

Exploration

June 23, 2026

1 Fase 0: Test scraping (Italia)

Prima di lanciare lo scraper su tutta l'Europa e rischiare il ban, facciamo un test sull'Italia per capire la logica del sito (Nowtricity). L'obiettivo è recuperare gli ID delle query nascoste nel frontend e scaricare i JSON senza far impazzire Selenium.

```
[ ]: from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
[ ]: from selenium import webdriver
from selenium.webdriver.chrome.options import Options

chrome_options = Options()
user_agent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36_
↳(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
chrome_options.add_argument(f"user-agent={user_agent}")

driver = webdriver.Chrome(options=chrome_options)

driver.set_page_load_timeout(20)

url_italy = "https://nowtricity.com/country/italy/"
print(f"{url_italy}")
driver.get(url_italy)
print(f"Page Title: {driver.title}")
```

```
[ ]: years = WebDriverWait(driver, 5).until(
    EC.presence_of_all_elements_located((By.CLASS_NAME, "js-load-year-data"))
)

for year in years:
    print(year.get_attribute("data-year"))
```

```
[ ]: import time

all_queries = []
```

```

for year_element in years:

    # Prendo l'anno dall'attributo, se vuoto lo peso dal testo
    year_str = year_element.get_attribute("data-year")
    if not year_str:
        year_str = year_element.text.strip()

    print(f"Estrazione dati per l'anno: {year_str}")

    # Click forzato via JS (il click standard di Selenium qui a volte sbrocca)
    driver.execute_script("arguments[0].click();", year_element)

    # Aspetto che il DOM carichi i mesi
    time.sleep(1)

    # Cerco i mesi dentro l'anno
    months = WebDriverWait(driver, 10).until(
        EC.presence_of_all_elements_located((By.CSS_SELECTOR,
        ↪f"[data-year='{year_str}'] .js-load-graph"))
    )

    # Salvo le query (es. '202501')
    for month in months:
        query = month.get_attribute("data-query")
        if query:
            all_queries.append(query)
            print(f" -> Trovata query: {query}")

print(f"\nEstrazione completata. Trovate {len(all_queries)} query totali.")

```

```

[ ]: import time
import random
import json
from selenium.webdriver.common.by import By

raw_data_list = []

print(f"Inizio estrazione per {len(all_queries)} query.")

for query_id in all_queries:
    ajax_url = f"https://www.nowtricity.com/ajax.php?
    ↪c=Pais&m=svg&country=italy&type=month&query={query_id}"

    try:
        driver.get(ajax_url)

```

```

# Il server sputa il JSON direttamente nel body, lo recupero grezzo
body_text = driver.find_element(By.TAG_NAME, "body").text

json_data = json.loads(body_text)

raw_data_list.append({
    "month": query_id,
    "json_payload": json_data
})
print(f"[{query_id}]    Dati scaricati con successo.")

except json.JSONDecodeError:
    print(f"[{query_id}]    ERRORE: Il server non ha restituito un JSON_
↪valido.")
    break # Fermo tutto se sbatte contro un blocco/errore server

except Exception as e:
    print(f"[{query_id}]    Errore di connessione o Selenium: {e}")
    break

# Pausa random per evitare di farsi bannare l'IP
wait = random.uniform(3.5, 6.5)
time.sleep(wait)

print(f"Raccolti {len(raw_data_list)} mesi su {len(all_queries)} totali.")

```

```

[ ]: import json

file_path = "../data/nowtricity_raw_italy.json"

with open(file_path, "w", encoding="utf-8") as f:
    json.dump(raw_data_list, f, ensure_ascii=False, indent=4)

```

2 1.0.1 Parsing HTML ed Export

Il JSON contiene un bel po' di HTML sporco. Qui usiamo BeautifulSoup per pulire i dati, formattiamo i nomi delle colonne in snake_case e buttiamo fuori un CSV di prova per vedere se ha senso.

```

[ ]: import json
import pandas as pd
from bs4 import BeautifulSoup

file_path = "../data/nowtricity_raw_italy.json"

try:
    with open(file_path, "r", encoding="utf-8") as f:

```

```

        # Lettura in sicurezza del file json
        f.seek(0)
        raw_data = json.load(f)
        print(f" File trovato: caricati {len(raw_data)} mesi in memoria.")
except FileNotFoundError:
    print(f" ERRORE: il file {file_path} non esiste.")
    raw_data = []

clean_records = []

for element in raw_data:
    month_id = element["month"]
    payload = element["json_payload"]
    html_content = payload.get("html", "") if isinstance(payload, dict) else ""

    soup = BeautifulSoup(html_content, "html.parser")

    try:
        # Macro Metriche
        co2_raw = soup.find("text", class_="emissions").text.strip()
        renewable_raw = soup.find("div", class_="emissions-graph-renewable").
↪text.strip()
        total_raw = soup.find("div", class_="emissions-graph-total").text.
↪strip()

        # Tolgo testo inutile dai numeri
        co2_clean = co2_raw.replace(" g", "").strip()
        renewable_clean = renewable_raw.replace("% renewable", "").strip()
        total_clean = total_raw.replace(" TWh total", "").strip()

        record = {
            "country": "italy",
            "month_id": month_id,
            "co2_g_kwh": float(co2_clean) if co2_clean.replace('.', '', 1).
↪isdigit() else None,
            "renewable_perc": float(renewable_clean) if renewable_clean.
↪replace('.', '', 1).isdigit() else None,
            "total_twh": float(total_clean) if total_clean.replace('.', '', 1).
↪isdigit() else None
        }

        # Dettaglio fonti energetiche
        energy_voices = soup.find_all("div", class_="entry")

        for voice in energy_voices:
            text = voice.text.strip()

```

```

        if "(" in text and "%" in text:
            parts = text.split("(")

            # Formato in snake_case per il db
            source_name = parts[0].strip().lower().replace(" ", "_")
            perc_val = parts[1].replace("%", "").replace("%", "").strip()

            # Prefisso colonna standardizzato in "source_"
            column_name = f"source_{source_name}_perc"
            record[column_name] = float(perc_val) if perc_val.replace('.', '')
↪'', 1).isdigit() else 0.0

        clean_records.append(record)

    except AttributeError:
        print(f"[{month_id}] Dati mancanti o formato errato. Salto il record.")
        continue

print(f"\n--- SCANSIONE COMPLETATA: {len(clean_records)} record validi ---")

# Generazione CSV
if clean_records:
    df = pd.DataFrame(clean_records)

    # Metto a 0 le fonti mancanti
    df.fillna(0, inplace=True)

    csv_name = "../outputs/nowtricity_italy_detailed.csv"
    df.to_csv(csv_name, index=False, encoding="utf-8")

    print(f" CSV di dettaglio salvato in: {csv_name}")
    print("\nAnteprima del Dataset:")
    print(df.head())
else:
    print(" Nessun dato estratto, CSV non creato.")

```